NestJS

Avec Typescript

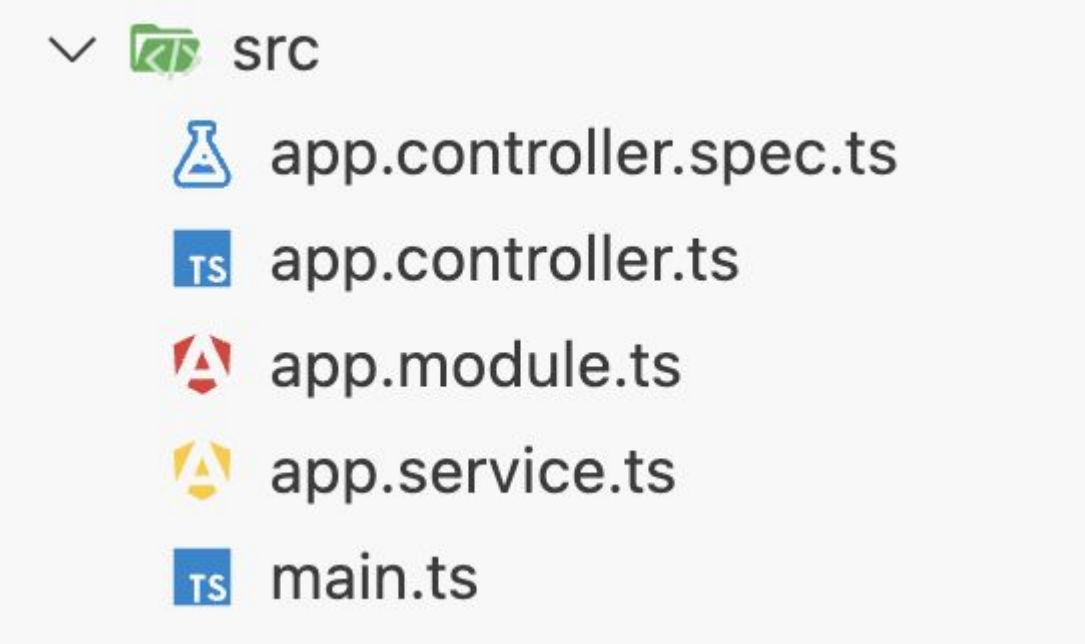
Installation

```
# Installation de la CLI
npm i -g @nestjs/cli

# Création du projet
nest new project-name

# Utilisation de la CLI sans installation
npx nest new project-name
```

Structure de base



- Tests unitaires pour le controller
- Le controller regroupe les routes
- Le module gère les ressources nécessaire
- Le service regroupe les règles métier

Controller

```
import { Controller, Get } from
'@nestjs/common';
@Controller('cats')
export class CatsController {
  @Get()
  findAll(): string {
    return 'This action returns all cats';
  }
}
```

@Controller et @Get sont des decorators

Routes de ce controller disponible sur /cats

Permet de déclarer à Nest la nature d'une classe ou d'une méthode

Controller - Gestion de la request

```
import { Controller, Get, Req } from '@nestjs/common';
import type { Request } from 'express';

@Controller('cats')
export class CatsController {
    @Get()
    findAll(@Req() request: Request): string {
        const headers = request.headers;
        const query = request.query;
        const params = request.params;
        const body = request.body;
        console.log(headers, query, params, body);

        return 'This action returns all cats';
    }
}
```

Injection de la requête dans les paramètres de la méthode.

Récupération de plusieurs infos utiles.

Controller - Récupération d'éléments précis de la request

```
import { Body, Controller, Post } from '@nestjs/common';

type body = {
    title: string;
}

@Controller('cats')
export class CatsController {
    @Post()
    titi(@Body() body: body): string {

        return 'This action returns all cats';
    }

    @Post()
    toto(@Body('title') title: string): string {

        return 'This action returns all cats';
    }
}
```

Injection du body entier ou récupération d'un élément précis du body.

Même logique applicable pour les decorators @Headers, @Query et @Params

Controller - Route avec paramètre dynamique

```
import { Controller, Get, Param } from '@nestjs/common';

@Controller('cats')
export class CatsController {
    @Get(':id')
    findOne(@Param('id') id: number): string {
        console.log(id);
        return `This action returns a #${id} cat`;
    }
}
```

Déclaration d'une route GET /cats/:id

Data Transfer Object - DTO

```
import { Body, Controller, Get, Param, Post } from
'@nestjs/common';
import type { CreateCatDto } from './create-cat.dto';

@Controller('cats')
export class CatsController {
    @Post()
    create(@Body() createCatDto: CreateCatDto): string {
        return 'This action adds a new cat';
    }
}
```

```
// create-cat.dto.ts
export class CreateCatDto {
    name: string;
    age: number;
    breed: string;
}
```

Services

```
import { Injectable } from '@nestjs/common';
import { Cat } from './interfaces/cat.interface';

@Injectable()
export class CatsService {
  private readonly cats: Cat[] = [];

  create(cat: Cat) {
    this.cats.push(cat);
  }

  findAll(): Cat[] {
    return this.cats;
  }
}
```

```
export interface Cat {
  name: string;
  age: number;
  breed: string;
}
```

Services - injection du service dans le controller

```
import { Controller, Get, Post, Body } from '@nestjs/common';
import { CreateCatDto } from './dto/create-cat.dto';
import { CatsService } from './cats.service';
import { Cat } from './interfaces/cat.interface';

@Controller('cats')
export class CatsController {
  constructor(private catsService: CatsService) {}

  @Post()
  async create(@Body() createCatDto: CreateCatDto) {
    this.catsService.create(createCatDto);
  }

  @Get()
  async findAll(): Promise<Cat[]> {
    return this.catsService.findAll();
  }
}
```

Services - déclaration dans les providers

```
import { Module } from '@nestjs/common';
import { CatsController } from './cats/cats.controller';
import { CatsService } from './cats/cats.service';

@Module({
  controllers: [CatsController],
  providers: [CatsService],
})
export class AppModule {}
```

Setup de l'environnement

- Ajout de Swagger https://docs.nestjs.com/openapi/introduction
- Ajout de Prisma https://docs.nestjs.com/recipes/prisma#set-up-prisma

Pour initialiser Prisma:

```
pnpx prisma init --datasource-provider postgresql --output ../generated/prisma
```

- Validation https://docs.nestjs.com/techniques/validation
- Authentification + AuthGuard https://docs.nestjs.com/security/authentication
- Update Swagger pour pouvoir utiliser le token https://docs.nestjs.com/openapi/security#bearer-authentication

Exo

- Instancier l'API Nest
- Faire un controller avec les différents endpoints GET, POST, PUT, DELETE
- Ajouter Swagger
- Ajouter Prisma
- Ajouter la création de compte et la connexion
- Ajouter le AuthGuard
- Ajouter sur swagger la possibilité de donner le token pour authentifier les calls à l'API

Helmet

Protège contre les vulnérabilités HTTP les plus connus en ajoutant des headers de sécurité.

https://github.com/helmetjs/helmet#how-it-works

SSE

Websockets

Microservices

Détacher l'authentification dans un microservice